

Diagnóstico Técnico — FCH AutoLab (partsbot)

Fecha: 2026-07-07

Repositorio: <https://github.com/fjcht/partsbot>

Alcance: Auditoría completa de código. **Sin cambios aplicados** — solo diagnóstico y análisis.

Objetivo principal: Explicar por qué la búsqueda por marca (ej. "Chery") no devuelve resultados aunque el servidor responde 200 OK.

1. Estructura del proyecto

```
partsbot/
├── main.py                # App FastAPI + endpoints (228 líneas)
├── models.py              # Modelos SQLAlchemy (52 líneas)
├── database.py            # Conexión PostgreSQL + get_db (33 líneas)
├── sincronizador.py       # Script que baja el catálogo de CassChoice (57 líneas)
├── scraper.py             # Consulta puntual query_commodity (parts + precios)
├── prueba.py              # Script de exploración de la API de vehículos
├── index.html             # Frontend (formulario + fetch + render de tabla)
├── merchant.txt           # ★ Volcado real de 3 respuestas de la API CassChoice
├── docker-compose.yml     # api (8000) + db (postgres:15-alpine)
├── Dockerfile             # imagen playwright/python
├── requirements.txt       # fastapi, sqlalchemy 2.0, psycpg2, httpx, gemini
├── .env                   # CASS_SID / CASS_TOKEN
└── (BUILD.txt, iniciar_partsbot.bat, appcustomnavbar.txt, scraper aux)
```

Stack confirmado: FastAPI + SQLAlchemy 2.0.31 + PostgreSQL 15 (Docker) + frontend HTML/JS vanilla. DB: `postgresql://postgres:admin123@{db|localhost}:5432/repuestos_db`.

2. TL;DR — Causa raíz

La búsqueda por marca **nunca puede devolver una autoparte** por **tres fallas encadenadas**:

1. **La tabla compatibilidades está vacía** (y autopartes también). El único sincronizador que corre (`/sincronizar_datos_completos`) intenta leer el OEM con `nodo.get("name")` , pero **ese campo NO existe** en la API de vehículos → `oem` siempre es `None` → nunca se crea ninguna Autoparte ni ninguna Compatibilidad . (Verificado sobre los 2798 nodos reales de `merchant.txt` : 2798/2798 con `oem = None` .)
2. **El endpoint /consultar NO hace el JOIN que dice hacer**. El comentario en `main.py:107` afirma “unimos Autoparte con Compatibilidad para filtrar por marca/modelo”, pero el código **solo consulta CatalogoVehiculos** filtrando por marca. Nunca cruza Autoparte ↔ Compatibilidad por `marca_vehiculo` . Por tanto una búsqueda por marca jamás llega a una pieza real.
3. **En una búsqueda solo-por-marca parte siempre queda en None** , así que el endpoint entra en la rama “Pieza no encontrada” (`main.py:119-127`) y responde `descripcion = "Pieza no encontrada para estos criterios"` , `precio = "0.00"` . El 200 OK es real, pero el payload no contiene

ninguna pieza. Si además `catalogo_vehiculos` está vacío, la lista `compatibilidades` sale vacía y el frontend muestra “No se encontraron vehículos compatibles.” → percepción de “no devuelve nada”.

En resumen: **el sistema tiene (a lo sumo) un catálogo de vehículos, pero CERO datos de piezas**, y el endpoint de marca ni siquiera está diseñado para ir de marca → pieza.

3. Análisis del flujo completo (frontend → backend → SQLAlchemy → JSON → render)

3.1 Frontend — `index.html`

- **Validación (`index.html:65`):** exige Marca o Número de Parte. Buscar solo “Chery” es válido.
- **Fetch (`index.html:82`):** GET `http://localhost:8000/consultar?marca=Chery` . URL absoluta a `localhost:8000` (ver §6, riesgo CORS/despliegue).
- **Parseo (`index.html:83`):** `const data = await res.json();` — correcto, no hay bug de parseo. El JSON que envía el backend tiene exactamente las llaves que el frontend lee (`oem` , `origen` , `descripcion` , `precio` , `disponibilidad` , `compatibilidades`).
- **Render (`index.html:97-103`):** recorre `data.compatibilidades` ; si viene vacío pinta “No se encontraron vehículos compatibles.”

✓ **El frontend NO es la causa.** El parseo y el render son correctos; simplemente reciben un payload sin piezas.

3.2 Backend — GET `/consultar` (`main.py:87-150`)

```

parte = None
if numero_parte:                                # línea 99-100
    parte = db.query(Autoparte).filter(numero_oem == numero_parte).first()
if not parte and pieza:                          # línea 103-104
    parte = db.query(Autoparte).filter(descripcion.ilike(...)).first()

query_vehiculos = db.query(models.CatalogoVehiculos) # línea 108
if marca:                                          # línea 111-112
    query_vehiculos = query_vehiculos.filter(CatalogoVehiculos.marca.ilike(f"%{marca}%"))
if modelo:                                       # línea 113-114
    query_vehiculos = query_vehiculos.filter(CatalogoVehiculos.modelo.ilike(f"%{modelo}%"))
vehiculos = query_vehiculos.all()                # línea 116

if not parte:                                   # línea 119 ← SIEMPRE en
búsqueda por marca
    return { ... "descripcion": "Pieza no encontrada para estos criterios", "precio": "
0.00",
            "compatibilidades": [ ...vehiculos... ] } # línea 120-127

```

Problemas concretos:

- **Línea 99-104:** `parte` solo se puede resolver por `numero_parte` o por `pieza` . **La marca nunca resuelve una pieza.** Como en la búsqueda por marca no hay `numero_parte` , `parte = None` → rama de “no encontrado”.

- **Línea 108-116 (el “JOIN” inexistente):** solo se filtra `CatalogoVehiculos` . **No hay** `join(models.Compatibilidad)` **ni filtro por** `Compatibilidad.marca_vehiculo` . El comentario de la línea 107 promete un JOIN que no está implementado.
- **Línea 134-137:** las compatibilidades “reales” (`parte.compatibilidades`) solo se leen si ya se encontró `parte` — algo imposible buscando por marca.
- **Sin filtro por año:** el parámetro `anio` se acepta (`main.py:91`) pero nunca se usa para filtrar (y `CatalogoVehiculos` ni siquiera tiene columna de año).

3.3 Respuesta JSON

Para `?marca=Chery` el backend devuelve (200 OK):

```
{
  "oem": "N/A", "origen": "N/A",
  "descripcion": "Pieza no encontrada para estos criterios",
  "precio": "0.00", "disponibilidad": "N/A",
  "compatibilidades": [ {"marca": "CHERY", "modelo": "", "desde": "", "hasta": "Actual"} ] /
  / solo si catalogo_vehiculos tiene Chery
}
```

- Si `catalogo_vehiculos` tiene la marca → aparece una fila (con `modelo` vacío) pero con el cartel “Pieza no encontrada”.
- Si `catalogo_vehiculos` está vacío → `compatibilidades: []` → frontend: “No se encontraron vehículos compatibles.”

3.4 Render final

El frontend pinta lo que reciba; con este payload el usuario **nunca ve una autoparte, ni código OEM, ni precio** → se percibe como “no devuelve resultados”.

4. Estado de la base de datos y relaciones entre tablas

4.1 Modelos (`models.py`)

Tabla	Columnas clave	Relaciones
<code>autopartes</code> (<code>Autoparte</code>)	<code>id</code> , <code>numero_oem</code> , <code>marca</code> , <code>modelo</code> , <code>descripcion</code>	1→N <code>compatibilidades</code> , 1→N <code>precios_casschoice</code> (cascade delete-orphan)
<code>compatibilidades</code> (<code>Compatibilidad</code>)	<code>id</code> , <code>autoparte_id</code> (FK) , <code>marca_vehiculo</code> , <code>modelo_vehiculo</code> , <code>anio_inicio</code> , <code>anio_fin</code>	N→1 <code>autoparte</code>
<code>precios_casschoice</code> (<code>PrecioCassChoice</code>)	<code>id</code> , <code>autoparte_id</code> (FK), <code>calidad</code> , <code>marca_repuesto</code> , <code>precio_fob</code> , <code>disponibilidad</code> , <code>ultima_actualizacion</code>	N→1 <code>autoparte</code>
<code>catalogo_vehiculos</code> (<code>CatalogoVehiculos</code>)	<code>id</code> , <code>marca</code> , <code>modelo</code>	(sin FK — tabla aislada)

Observaciones estructurales:

- `compatibilidades` se relaciona con `autopartes` por `autoparte_id` , pero **guarda marca/modelo como texto libre** (`marca_vehiculo` , `modelo_vehiculo`) en vez de referenciar `catalogo_vehiculos` por FK. → Modelo de datos desnormalizado y sin integridad referencial hacia el catálogo.
- `catalogo_vehiculos` es una **isla**: no tiene columna de año y no se une con nada. Los años (que sí existen en la API) se pierden.
- `precios_casschoice.precio_fob` es el precio FOB **crudo** — no hay lógica de margen 6% en ninguna parte del código (requisito de negocio pendiente).

4.2 Estado real de los datos (inferido del código + `merchant.txt`)

- **`catalogo_vehiculos` : poblada solo si** se corrió `sincronizador.py` (que llama a `/sincronizar_datos_completos`). Ese endpoint sí crea filas de catálogo por cada nodo (`marca= make` , `modelo= model`). Con los datos reales serían **2798 filas** (129 marcas + modelos + años como nodos), muchas con `modelo=""` .
- **`autopartes` : VACÍA.** Ningún endpoint activo la puebla (ver §5). Las funciones `obtener_datos_cass()` y `guardar_en_db()` (`main.py:50-83`) que sí traerían piezas+precios reales **están definidas pero nunca se llaman** desde ningún endpoint.
- **`compatibilidades` : VACÍA.** Depende de `item.get("oem")` , que siempre es `None` (§5).
- **`precios_casschoice` : VACÍA** (se llena dentro de `guardar_en_db` , nunca invocado).

⚠ El “2798” que el usuario asocia a compatibilidades corresponde en realidad a los **2798 nodos del árbol de vehículos** (`node_name` aparece 2798 veces en `merchant.txt`). Se cargaron como `catalogo_vehiculos` , **no** como compatibilidades de piezas.

5. Análisis del código de sincronización con CassChoice

`merchant.txt` contiene el volcado real de **tres** respuestas distintas de la API de CassChoice (verificado parseando el archivo):

Blob	Endpoint origen	message.data	Contenido	Campos
1	<code>list_vehicle_re lations</code>	lista de 129 marcas (árbol, 2798 nodos)	Catálogo de vehículos make/model/ year	<code>vehicle_relatio n_id, make, model, year, node_name</code>
2	categorías de producto	lista de 18	Taxonomía de categorías de commodity	<code>level, label, value, children</code>
3	lista de marcas de producto	lista de 387	Marcas de re- puestos	<code>brand_code, brand_name, brand_type</code>
—	<code>query_commodity (scraper.py)</code>	<code>data.results[]. products[]</code>	Piezas reales + precios	<code>parts_number, brand_name, brand_type, price.prices[{c urrency, de- fault_price}]</code>

5.1 Bug crítico en `sincronizador.py` (campo OEM inexistente)

```
# sincronizador.py:29-39
datos_a_enviar.append({
    "marca": nodo.get("make"),
    "modelo": nodo.get("model") or "",
    "pieza_descripcion": nodo.get("node_name"),
    "oem": nodo.get("name")          # ❌ el nodo NO tiene 'name'; el id es
    'vehicle_relation_id'
})
```

Los nodos reales tienen las llaves `['vehicle_relation_id', 'make', 'model', 'year', 'node_name']`. **No existe `name`**.

Verificación empírica sobre `merchant.txt`: de **2798** nodos, **2798** tienen `oem = None` (0 con OEM). Consecuencia directa en el backend:

```
# main.py:198
if item.get("oem"):                # ❌ SIEMPRE False → no se crea Autoparte ni Com-
    patibilidad                    patibilidad
    autoparte = db.query(models.Autopartes)... # (ver bug §5.2)
    ...
    compat = models.Compatibilidad(...)      # nunca se ejecuta
```

➡ **Por esto la tabla `compatibilidades` (y `autopartes`) queda vacía.** Además, conceptualmente el endpoint de vehículos **no trae piezas ni OEM**; los OEM/precios viven en `query_commodity` (blob de piezas), que la sincronización actual nunca consulta.

5.2 Bug de nombre de clase en `/sincronizar_datos_completos` (`main.py:199-201`)

```

autoparte = db.query(models.Autopartes).filter_by(numero_oem=item["oem"]).first() #
❌ 'Autopartes'
autoparte = models.Autopartes(numero_oem=item["oem"], descripcion=...) #
❌ 'Autopartes'

```

El modelo se llama **Autoparte** (singular, `models.py:6`). `models.Autopartes` no existe → lanzaría `AttributeError` (HTTP 500) si alguna vez `item.get("oem")` fuera verdadero. Hoy queda enmascarado porque nunca se entra al `if` (§5.1).

5.3 Bug en `/sincronizar_catalogo` (`main.py:153-165`)

```

datos = resp.json().get('message', []) # ❌ 'message' es un DICT
{code,message,data}, no una lista
for item in datos: # itera las CLAVES del dict:
    'code','message','data' (str)
    db.add(models.CatalogoVehiculos(marca=item.get('brand'), ...)) # ❌ str.get → At-
tributeError

```

`message` es un objeto con `code/message/data`; iterarlo recorre strings y `item.get('brand')` explota. Este endpoint automático está roto. (El correcto sería `.get('message',{}).get('data',[])`, tal como sí hace `sincronizador.py:25`.)

5.4 Inconsistencia de nombres de campo entre sincronizadores

- `/sincronizar_catalogo` usa `item.get('brand')` / `item.get('model')`.
- `/sincronizar_catalogo_manual` usa `item.get('make')` / `item.get('model')`.
- `/sincronizar_datos_completos` usa `item["marca"]` / `item["modelo"]`.
- La API real usa `make` / `model`. No hay un contrato único → fuente de errores.

5.5 Credenciales / seguridad

- `scraper.py:6-7` tiene `TOKEN` / `SID` **hardcodeados** (además de estar en `.env`). El `.env` está versionado en el repo. **Recomendado rotar credenciales** y sacarlas del control de versiones.

6. Otros hallazgos relevantes

- **CORS / URL fija:** el frontend llama a `http://localhost:8000` absoluto (`index.html:82,111`). Sirviendo el HTML desde otro origen/host, `allow_origins=["*"]` ayuda pero la URL fija rompe en producción. Conviene usar ruta relativa o variable de entorno.
- **obtener_info_ia con Gemini (`main.py:38-47`):** modelo `'gemini-pro'` (deprecado en la versión reciente del SDK) y **API key hardcodeada** (`main.py:20`). Función tampoco se usa en el flujo de consulta.
- **Precio:** no existe aplicación del **margen de 6%** sobre FOB en ningún punto (requisito de negocio).

- **Búsqueda bilingüe (ES/EN):** no implementada; las descripciones vienen de `node_name` (inglés) sin traducción/normalización.
- **Códigos múltiples (OE/OEM/aftermarket):** el modelo solo guarda `numero_oem` (un código). No hay soporte para múltiples números por pieza (requisito de negocio).

7. Recomendaciones de arquitectura (para las mejoras solicitadas)

Estas son propuestas; **no se han aplicado cambios**.

7.1 Arreglos mínimos para desbloquear la búsqueda por marca

1. **Poblar datos de piezas de verdad:** cablear `obtener_datos_cass()` + `guardar_en_db()` a un endpoint/flow de sincronización que consulte `query_commodity` con números de parte (única fuente de OEM + precios + marca de repuesto).
2. **Implementar el JOIN real** en `/consultar` para marca/modelo:
`db.query(Autoparte).join(Compatibilidad).filter(Compatibilidad.marca_vehiculo.ilike(f"%{marca}%"))` (+ filtro por año con `anio_inicio/anio_fin`). Hoy ese JOIN no existe.
3. **Corregir `sincronizador.py`**: el árbol de vehículos alimenta `catalogo_vehiculos` (make/model/year), **no** compatibilidades. Las compatibilidades deben derivarse del cruce pieza↔vehículo que provee la API de commodity, no del `nodo.get("name")` inexistente.
4. **Renombrar `models.Autopartes` → `models.Autoparte`** (`main.py:199-201`) y arreglar `/sincronizar_catalogo` (`.get('message', {})`).get('data', [])).

7.2 Modelo de datos propuesto (re-sincronización desde cero)

- **`autopartes`**: `id, sku_interno, descripcion_es, descripcion_en, categoria, ...`.
- **`codigos_parte`** (NUEVA, 1→N con autopartes):
`autoparte_id, tipo (OE/OEM/AFTERMARKET), codigo, marca_repuesto` → soporta **múltiples códigos** por pieza.
- **`catalogo_vehiculos`**: agregar `anio` (o mantener árbol make/model/year) y usarlo como catálogo canónico.
- **`compatibilidades`**: referenciar `catalogo_vehiculos` por **FK** (o al menos normalizar marca/modelo) y agrupar por **rango de años** (`anio_inicio`, `anio_fin`) para el requisito “compatibilidades agrupadas por rangos”.
- **`precios_casschoice`**: guardar `precio_fob` y `precio_venta = precio_fob * 1.06` (margen 6%), con `moneda` y `ultima_actualizacion`.

7.3 Sincronización robusta desde CassChoice

- Fase 1: `list_vehicle_relations` → `catalogo_vehiculos` (con año).
- Fase 2: categorías (blob 2) + marcas (blob 3) como tablas de referencia.
- Fase 3: `query_commodity` por lotes de `parts_number` → `autopartes` + `codigos_parte` + `precios_casschoice` (+ margen) + `compatibilidades`.
- Idempotencia: upsert por código en vez de `delete()` masivo; logging de errores por lote.
- Mover credenciales a `.env` **fuera** del repo y **rotarlas**.

7.4 Frontend (evolución a tienda B2B)

- URL de API relativa/configurable (no `localhost:8000` fijo).

- Vista de resultados como **listado de piezas** (código OE/OEM/aftermarket, marca, calidad, precio con margen), con panel de compatibilidades agrupadas por rango de años.
- Búsqueda bilingüe: normalizar/traducir términos ES↔EN antes de consultar.

8. Índice de líneas de código citadas

Archivo:línea	Hallazgo
<code>sincronizador.py:38</code>	<code>oem = nodo.get("name")</code> → campo inexistente → 2798/2798 en <code>None</code>
<code>main.py:198</code>	<code>if item.get("oem")</code> nunca <code>True</code> → <code>compatibilidades / autopartes</code> vacías
<code>main.py:199-201</code>	<code>models.Autopartes</code> no existe (debe ser <code>Autoparte</code>)
<code>main.py:107-116</code>	“JOIN” prometido pero no implementado; solo filtra <code>CatalogoVehiculos</code>
<code>main.py:99-104</code>	<code>parte</code> solo se resuelve por <code>numero_parte / pieza</code> , nunca por marca
<code>main.py:119-127</code>	Búsqueda por marca cae siempre en “Pieza no encontrada” (200 OK)
<code>main.py:159-162</code>	<code>/sincronizar_catalogo</code> itera un dict como lista → roto
<code>main.py:50-83</code>	<code>obtener_datos_cass / guardar_en_db</code> definidos pero nunca invocados
<code>models.py:44-53</code>	<code>catalogo_vehiculos</code> sin año y sin FK (tabla aislada)
<code>index.html:82-103</code>	Fetch/parseo/render correctos; reciben payload sin piezas

Conclusión: El 200 OK es legítimo, pero el diseño actual **no puede** entregar una autoparte en una búsqueda por marca porque (1) no hay datos de piezas cargados, (2) las tablas `autopartes / compatibilidades` están vacías por el bug del campo `oem`, y (3) el endpoint `/consultar` no implementa el JOIN marca→compatibilidad→autoparte que su propio comentario promete. La corrección requiere re-sincronizar piezas desde `query_commodity`, normalizar el modelo de datos e implementar el JOIN real.